

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Manoj M	Reg. No.:	192511060
Section / Batch:	CSE	Date:	27.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NPComplete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Minimum Linear Arrangement problem:

1. **Define the Problem:** Arrange graph vertices in a linear order to **minimize the maximum edge length**.
2. **Define Constraints:** Clearly state the rules for:
 - **Vertex ordering** (each vertex gets a unique position).
 - **Edge distance** (calculating the distance between connected vertices).
3. **Analyze Impact:** Discuss how these constraints affect which arrangements are possible (feasible).
4. **Develop a Solution:** Create a **backtracking algorithm** to solve this.
5. **Apply Pruning:** Implement techniques to cut off unproductive search paths and reduce the search space.

Code of Conduct

I _____ Manoj (192511060)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				

Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #define MAXV 100
5
6 int V, E;
7 int adj[MAXV][MAXV];
8 int degree[MAXV];
9
10 int pos[MAXV];           // Maps vertex -> layout index
11 int layout[MAXV];        // Array storing the linear layout
12 int used[MAXV];          // Tracks placed vertices
13
14 int min_bandwidth;
15 int best_layout[MAXV];
16
17 void backtrack(int idx, int current_max_b) {
18     // Pruning 1: Current bandwidth exceeds or equals best known bandwidth
19     if (current_max_b >= min_bandwidth) {
20         return;
21     }
22
23     // Base Case: All vertices placed
24     if (idx == V) {
25         min_bandwidth = current_max_b;
26         for (int i = 0; i < V; i++) {
27             best_layout[i] = layout[i];
28         }
29         return;
30     }
31
32     // Try placing vertex idx at each position
33     for (int i = 0; i < V; i++) {
34         if (!used[i]) {
35             // Place vertex idx at position i
36             pos[i] = idx;
37             layout[i] = idx;
38             used[i] = 1;
39
40             // Calculate current bandwidth
41             int current_max_b = 0;
42             for (int j = 0; j < V; j++) {
43                 if (adj[i][j] > 0) {
44                     current_max_b = fmax(current_max_b, abs(pos[i] - pos[j]));
45                 }
46             }
47
48             // Recurse
49             backtrack(idx + 1, current_max_b);
50
51             // Remove vertex idx from position i
52             pos[i] = -1;
53             layout[i] = -1;
54             used[i] = 0;
55         }
56     }
57 }
```

Output:

Output

```
Minimum Bandwidth: 2
Optimal Linear Layout: 0 1 3 2
```

Algorithm:

Constraints

1. Vertex Ordering:

- Each vertex must be assigned a unique position in $\{1, 2, \dots, n\}$.
- Formally: $p(u) \neq p(v)$ for $u \neq v$, and p is a bijection.

2. Edge Distance:

- For every edge (u, v) , the distance $|p(u) - p(v)|$ must be computed.
- The objective is to minimize the maximum of these distances.

3. Feasibility:

- A valid arrangement must satisfy all constraints (unique positions, valid distances).

Impact of Constraints on Feasible Arrangements

- Vertex Ordering: Forces a permutation of vertices, drastically reducing the search space from infinite to $n!$ possibilities.
- Edge Distance: Directly ties the geometric layout to the graph's topology. Denser graphs (more edges) often require larger maximum edge lengths due to conflicting distance requirements.
- Trade-offs: Minimizing the maximum edge length may force some edges to have larger lengths elsewhere, creating a balancing act.

Backtracking-Based Solution

Idea: Build the arrangement incrementally, assigning positions one vertex at a time. At each step, check if the partial assignment violates constraints or can be pruned.

Algorithm Steps

1. Initialization:

- Start with an empty assignment p .
- Track used positions (initially empty).

2. Recursive Backtracking:

- Base Case: If all vertices are assigned, compute the maximum edge length and update the global minimum.
- Recursive Step:
 - For the current vertex u , try assigning it to an unused position $k \in \{1, \dots, n\}$.
 - Constraint Check:
 - Ensure k is unused.
 - For all neighbors v of u already assigned, compute $|k - p(v)|$. If this exceeds the current best maximum edge length, prune this branch.
 - Recurse for the next vertex.
 - Backtrack: Unassign u and try the next position.

3. Pruning:

- Dominance Pruning: If the current partial assignment already has a maximum edge length $\geq$ the best found so far, stop exploring this branch.
- Symmetry Breaking: Fix the position of the first vertex (e.g., $p(v_1) = 1$) to avoid symmetric solutions.